

LLM 预训练之 Optimizer

AdamW · Optimizer State · Update Dynamics · Distributed Optimizer

深度研究手册 / 2026-08

Optimizer Step: 从梯度到参数更新

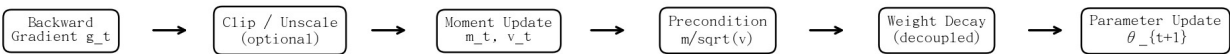


图 1 Optimizer Step 位于 Backprop 与 Learning-Rate Schedule 的交汇处

核心判断

Backprop 只告诉我们每个参数当前的梯度；Optimizer 决定如何利用历史梯度、尺度信息、正则化与参数几何，把这一瞬时信号转换成真正的参数位移。对于 LLM，Optimizer 同时也是显存、通信、checkpoint 和数值稳定性的系统组件。

边界	负责什么	不负责什么
Backpropagation	计算当前 step 的梯度 g_t	不决定最终 step size
Optimizer	把 $g_t + \text{history}$ 转成 update $\Delta\theta_t$	通常不决定全局 LR 随训练时间怎样变化
LR Scheduler	生成 $\eta_t / \text{warmup} / \text{decay}$	不改变 optimizer 的基本预条件机制
Gradient Clipping	限制进入 optimizer 的梯度尺度	不是 optimizer moment update 本身

一句话心智模型：Gradient 是“当前方向”；Momentum 是“时间滤波”；Adam 的 second moment 是“坐标级尺度归一化”；Weight Decay 是“参数尺度动力学”；Distributed Optimizer 则负责把这些状态放得下。

1. 从 SGD 到 Adam : Optimizer 到底改变了什么？

设第 t 个 optimizer step 的 mini-batch 梯度为 $g_t = \nabla L_t(\theta_t)$ 。最朴素的 SGD 直接执行 $\theta_{t+1} = \theta_t - \eta_t g_t$ 。现代 LLM optimizer 的本质是在这个更新前增加“时间滤波”和“坐标/矩阵预条件”。

方法	状态	核心更新直觉	主要优势	主要代价
SGD	无	直接沿 $-g_t$ 移动	状态最少、机制透明	尺度和敏感
Momentum	m	对历史梯度做 EMA	降低梯度噪声、形成惯性	多 1 个逐参数状态
Adam	m, v	$m / \text{sqrt}(v)$ 做坐标级自适应	对异质尺度/稀疏梯度友好	多 2 个逐参数状态
AdamW	$m, v + \text{decoupled WD}$	Adam + 独立参数衰减	现代 LLM 稳健基线	显存/状态仍大
Matrix-aware	矩阵级统计/正交化	利用矩何而非逐元素尺度	潜在更高 token/compute efficiency	kernel/通信/调参复杂

1.1 Momentum : 把 noisy gradient 变成低通滤波后的方向

常见形式： $m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t$ 。 β_1 越大，optimizer 越依赖更久之前的梯度；但响应当前非平稳梯度的速度也越慢。

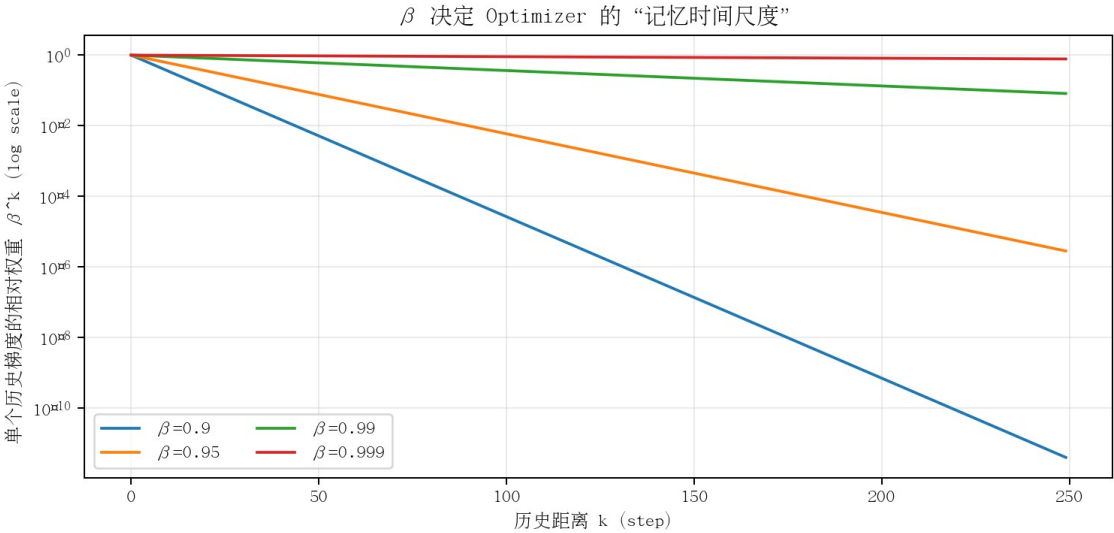


图 2 β 的时间尺度。注意： $\beta=0.999$ 的 second-moment memory 于 $\beta=0.95$ 。

β	$1/(1-\beta)$ 近似时间尺度	半衰期 (steps)
0.9	10.0	6.6
0.95	20.0	13.5
0.99	100.0	69.0
0.999	1000.0	692.8

Know-why

β_1/β_2 不是“神秘默认值”，而是两条不同的时间滤波器。 β_1 决定方向记忆， β_2 决定尺度估计记忆；数据分布、batch size、训练

horizon 或梯度统计发生变化时，固定 β 未必永远最优。

1.2 Adam : momentum + diagonal preconditioning

Adam 维护一阶矩 m_t 与二阶矩 v_t ： $m_t = \beta_1 m_{t-1} + (1-\beta_1)g_t$ ； $v_t = \beta_2 v_{t-1} + (1-\beta_2)g_t^2$ 。随后进行 bias correction，并以 $m_{\text{hat}}/(\text{sqrt}(v_{\text{hat}})+\epsilon)$ 形成逐坐标预条件更新。[R1]

- Bias correction 很重要： $m_0=v_0=0$ ，使训练早期的 moment 被系统性低估；除以 $1-\beta^t$ 修正这一初始化偏差。
- ϵ 不只用于“防除零”：当 $\text{sqrt}(v)$ 与 ϵ 同量级时， ϵ 会改变小方差参数的有效学习率，因此不同实现中的 ϵ 位置也可能造成差异。
- Adam 的优势不是“知道真正曲率”，而是用对角二阶统计对不同坐标的梯度尺度做自适应归一化。

2. AdamW : 为什么 Weight Decay 必须与 Adam 解耦？

L2 Regularization 与 Decoupled Weight Decay 在 Adam 中并不等价

Adam + L2 penalty

AdamW (Decoupled WD)

$$g \leftarrow \nabla L(\theta) + \lambda \theta$$

$$g \leftarrow \nabla L(\theta)$$

m, v 同时吸收 $\lambda \theta$

先做 Adam adaptive update

adaptive preconditioner 会改变 regularization

再独立做 $\theta \leftarrow (1-\eta \lambda) \theta$

图 3 Adam + L2 与 AdamW 的关键区别。左侧 regularization 会进入 m/v ；右侧 decay 不进入 moment。

在普通 SGD 中，L2 penalty 与 weight decay 在适当缩放下可以等价；但在 Adam 等 adaptive optimizer 中，把 $\lambda\theta$ 加进 gradient 后，它也会被 second-moment preconditioner 重标定，所以不再等价。AdamW 的关键贡献是把 θ 的衰减从 loss-gradient 更新中解耦。[R2]

AdamW 的典型更新（概念式）

1) 用纯任务梯度更新 m/v ；2) 做 adaptive update： $\theta \leftarrow \theta - \eta \cdot m_{\text{hat}}/(\text{sqrt}(v_{\text{hat}})+\epsilon)$ ；3) 独立做参数衰减： $\theta \leftarrow \theta - \eta \cdot \lambda \cdot \theta$ 。实际 fused kernel 可以重排计算，但语义应保持 decoupled。

2.1 Weight decay 的真正作用不仅是“防过拟合”

在 LayerNorm/RMSNorm 广泛存在的 scale-invariant 架构里，参数范数还会影响“同样一个参数更新对应多少角度变化”。近期关于 rotational equilibrium 的研究显示，learning rate、weight decay、initialization 与 normalization 会共同决定参数向量的稳态尺度与平均旋转幅度。[R8]

参数/机制	常做法	为什么要单独考虑
Matrix weights	通常应用 WD	控制重尺度；有效 update 共同形成稳态
Bias	多 recipe 不 decay	低维 shift 参数的 regularization 不同；但不是普适定律
Norm scale	常被排除或单独设置	scale 参数直接影响归一化后的增益

参数/机制	常做法	为什么要单独考虑
Embedding / LM head	可能独立 LR/WD	大词表、频率长尾、tied/untied 使统计不同
MoE experts	可用独立 param group	expert token count $\neq$ dense 参数不同

2.2 一公的大模配方：DeepSeek-V3

DeepSeek-V3 报告其预训练使用 AdamW， $\beta_1=0.9$ 、 $\beta_2=0.95$ 、weight_decay=0.1，并使用 gradient clip norm=1.0；更值得注意的是，其 optimizer 的 first/second moments 用 BF16 存储，但 master weights 与用于 batch accumulation 的 gradients 保持 FP32，以降低 optimizer memory 而维持数值稳定。[R14]

不要把配方数字当成定律

$\beta=(0.9,0.95)$ 、WD=0.1 在许多现代 LLM recipe 中常见，但最优值依赖模型参数化、batch size、训练长度、LR schedule、数据与数值精度。正确做法是用 proxy scaling experiments / μP / 小规模 sweep 做可迁移标定。

3. Optimizer State：为什么 AdamW 会成为显存大户？

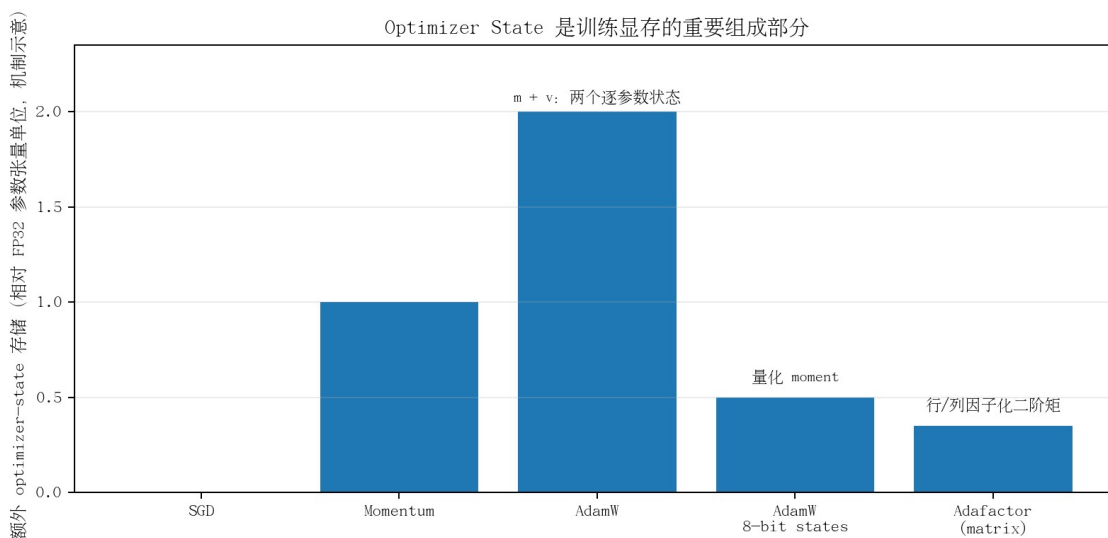


图 4 Optimizer-state 机制示意。柱高用于比较状态结构，不代表某个框架的固定 bytes/parameter。

AdamW 至少需要 m 和 v 两个与参数同形状的状态。混合精度训练还经常维护 FP32 master parameter 与更高精度 gradient buffer，因此训练显存远大于“模型权重文件大小”。具体 bytes/parameter 取决于 param/grad/state dtype 与框架实现。

方案	状态设计	内存 know-how	证据/备注
AdamW FP32 states	每参数 m + v	简单可靠；状态规模 $O(N)$	PyTorch/主流基线 [R12]
Adafactor	矩阵二阶矩按 row/column 因子化	把矩阵 v 从 $O(nm)$ 降为近 $O(n+m)$	经典 Transformer 实验 [R3]
8-bit Optimizer	m/v 做 block-wise 量化	显著压缩 optimizer state	1.5B LM 等实验保持 32-bit 性能 [R6]
Distributed Optimizer	state 沿 DP ranks 分片	用通信换显存；类似 ZeRO-1 思路	Megatron 当前实现 [R13]
Low-precision moments	m/v 降到 BF16 等	减少 state bandwidth + memory	DeepSeek-V3 公开实践 [R14]

3.1 ZeRO / Distributed Optimizer : optimizer 算法不变，状态布局改变

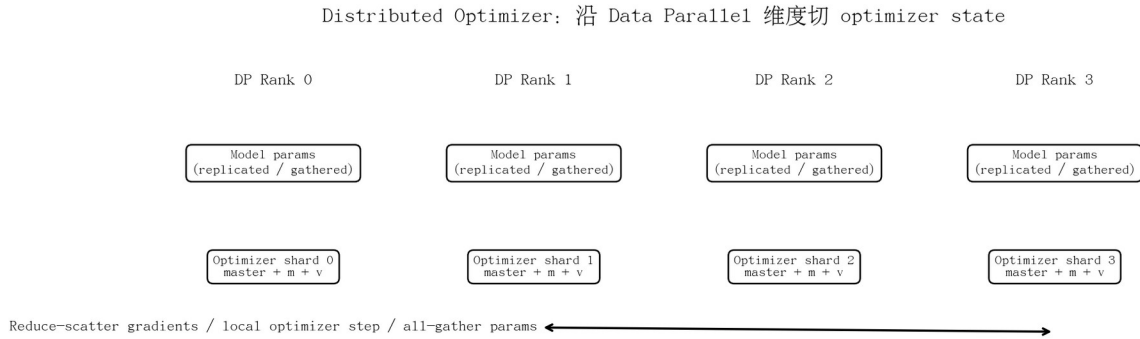


图 5 Distributed Optimizer 机制：梯度完成后切分到 owner rank，本地更新 shard，再按需 gather。

Megatron Core 的 distributed optimizer 将 optimizer states 沿 data-parallel ranks 分片，并给出不同 dtype 组合下的理论 bytes/parameter；其目的不是修改 AdamW 数学，而是避免每个 DP rank 都复制完整 master/m/v。[R13] 这一思想来自 ZeRO 对 optimizer state、gradient、parameter 冗余的分阶段消除。[R5]

- Checkpoint 也必须保存 optimizer states：只保存 model weights 无法无缝恢复 m/v、step counter、bias correction 与 scheduler phase。
- DP size 改变时需要 reshard：现代 checkpoint format 要支持 state 从旧 DP topology 映射到新 topology。
- State sharding 的代价是 reduce-scatter/all-gather 以及更复杂的 buffer ownership；因此需要与 backward/forward overlap。

3.2 Fused / foreach / kernel maturity

PyTorch 当前 AdamW 同时提供 for-loop、foreach 与 fused 实现；官方文档指出 fused 理论上拥有更高的垂直/水平融合潜力，而 foreach 通常比逐 tensor for-loop 更快但会消耗额外峰值内存。[R12] 在大模型训练中，optimizer step 本身也必须被视为 GPU kernel pipeline。

实现选择	优势	风险/代价
for-loop	最透明、内存中间量少	kernel launch 多、吞吐低
foreach	批量 tensor 操作、通常更快	额外 TensorList intermediates
fused	减少 launch / memory traffic	实现成熟度、dtype/graph capture 限制
CPU offload	GPU memory 更低	PCIe/NVLink 传输、CPU update latency
precision-aware state	减少 optimizer memory/bandwidth	需验证量化/舍入不会改变收敛

4. Update Dynamics : 真正应该监控的不是 Gradient Norm 一个数

同一个 gradient，在不同 optimizer、 β 、 ϵ 、weight decay 与参数尺度下，会产生完全不同的 parameter update。预训练 observability 因此必须同时记录 gradient、preconditioned update 与 parameter norm。

指标	定义/近似	诊断意义
Gradient RMS	RMS(g)	当前 batch 的原始监督强度

指标	定义/近似	诊断意义
Moment RMS	$\text{RMS}(m), \text{RMS}(\text{sqrt}(v))$	方向记忆与尺度估计是否异常
Update RMS	$\text{RMS}(\Delta\theta)$	正入的步幅
Update/Weight Ratio	$\ \Delta\theta\ / \ \theta\ $	不同层的相对更新速度
Angular Update	$\Delta\theta \perp \theta$ 的方向变化	scale-invariant 参数的有效学习强度
Grad-Update Cosine	$\cos(g, \Delta\theta)$	optimizer preconditioning 改变方向多少
WD/Adaptive Ratio	$\ \eta\lambda\theta\ / \ \text{adaptive update}\ $	weight decay 是否主导某些层
State dtype saturation	m/v 的 under/overflow/quant error	低精度 optimizer 的稳定性

关键 know-how

Global grad norm 正常，不代表所有层都正常。一个稳定的大模型训练 run 应同时观察 per-layer / per-parameter-group 的 update-to-weight ratio、m/v 统计、weight norm 和 WD contribution。很多“突然 loss spike”在全局指标里出现之前，局部 update dynamics 已经发生漂移。

4.1 β_2 : 稳定性与响应速度的 trade-off

β_2 越高， v_t 越平滑，噪声越小，但对梯度方差突然变化的响应越慢。LLM 中常见 $\beta_2=0.95/0.98/0.99/0.999$ 等差异，本质上是对 second-moment memory horizon 的选择。固定 β_2 并不保证在 warmup、data phase transition、long-context extension 或 annealing 阶段都同样合适。

4.2 ϵ : 小参数/小方差坐标的隐形学习率控制器

当 $\text{sqrt}(v_{\text{hat}}) \gg \epsilon$ 时， ϵ 几乎无影响；但在低方差坐标上，denominator 被 ϵ 主导，adaptive behavior 会弱化。因此复制 optimizer recipe 时需要连同 ϵ 与其放置位置一起复制，而不能只复制 β 和 LR。

5. AdamW 之外：为什么 2023-2026 Optimizer 研究重新活跃？

LLM Optimizer 方法谱系

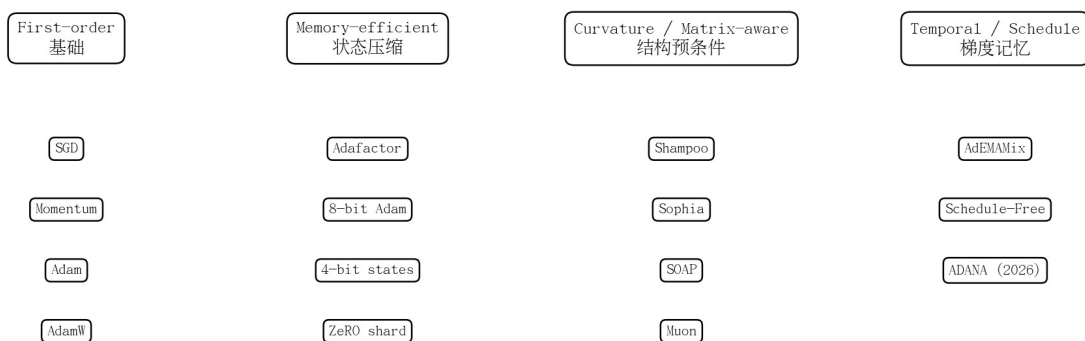


图 6 LLM Optimizer 方法谱系。右侧 2025-2026 方法应视作快速演进中的前沿，而非统一生产共识。

方法	核心 idea	LLM 证据	主要 caveat
Adafactor [R3]	因子化二阶矩 + update clipping	Transformer 训练可接近 Adam , state 更小	与标准 AdamW recipe 不完全同构
LAMB [R4]	Adam-style update + layerwise trust ratio	BERT 大 batch 32K+ 的经典结果	今天 decoder pretrain 不一定需要
Sophia [R7]	稀疏估计 diagonal Hessian + clipping	125M-1.5B GPT 报告约 2× steps/compute efficiency	需要 Hessian estimator 与额外实现
SOAP [R9]	在 Shampoo eigenbasis 中运行 Adam	360M/660M LM 大 batch : steps >40%、wall time >35% 减少	preconditioner/eigendecomp 开销与规模验证
Muon [R10]	matrix update 做 Newton-Schulz orthogonalization	Moonlight 3B/16B MoE、5.7T tokens ; 论文报告约 2× compute efficiency	通常只用于 2D matrix 参数，需混合 Adam 处理其他参数
AdEMAMix [R11]	快 EMA + 慢 EMA 混合	1.3B LM : 101B tokens 对比 AdamW 197B 的报告	多一尺度量
ADANA 2026 [R17]	β /WD 随 log-time 调度	45M-2.6B 报告最高约 40% compute efficiency gain	2026 新预印本，需更多独立复现

公平比较原则

Optimizer claim 必须同时比较：validation loss vs tokens、loss vs FLOPs、loss vs wall-clock、peak memory、hyperparameter tuning budget、kernel maturity、batch size、model scale、training horizon。只比较 steps-to-loss 往往会把每-step 额外开销藏起来。2025 的 standardized optimizer benchmark 正是针对这一问题。[R16]

5.1 Muon 为什么代表了一个不同的方向？

Adam 是逐元素（coordinate-wise）预条件；Muon 把 2D weight matrix 当作整体，通过矩阵正交化塑造 update 的 singular spectrum。Moonshot 的大规模研究指出，Muon 展到 LLM 需要两项关键修正：加入 weight decay，以及仔细设置 per-parameter update scale；并用分布式实现训练 5.7T-token MoE 模型。[R10]

- 现代框架已经开始支持“混合 optimizer”：例如 Megatron 当前 optimizer factory 可将 linear weights 交给 Muon，而其余参数走 Adam/SGD param groups。
- 这提示未来 optimizer 可能不再是“全模型一个算法”，而是 parameter geometry-aware 的组合策略。

6. Production Optimizer Blueprint

阶段	关键动作	必须记录的状态
1. Parameter grouping	按 matrix/vector、norm/bias、embedding/head、expert/dense 划组	param name、shape、group id、LR/WD multiplier
2. Gradient preparation	unscale、NaN/Inf check、global/per-group clip	grad norm、overflow flag、valid step
3. Moment update	更新 m/v 或 matrix state	β 、state dtype、step counter

阶段	关键动作	必须记录的状态
4. Preconditioning	形成 adaptive / matrix-aware update	update RMS、condition / spectral stats
5. Weight decay	按 group 做 decoupled decay	WD contribution ratio
6. Parameter write	master weight update + cast/sync	param norm、update/weight ratio
7. Distributed sync	RS/AG 或 owner-rank state update	communication bytes、overlap time
8. Checkpoint	持久化 param + optimizer + scaler + scheduler	schema version、DP topology、state dtype

6.1 推荐的最小 Observability Dashboard

层级	指标
全局	train loss、LR、grad norm、clip fraction、overflow/skipped step、optimizer-step time
parameter group	weight norm、update norm、update/weight ratio、WD/adaptive ratio、m RMS、v RMS
layer	最大/最小 update ratio、异常层排名、embedding/head 与 backbone 的相对 LR
systems	optimizer-state GB、RS/AG time、CPU offload bytes、fused-kernel utilization
checkpoint	state completeness、step continuity、resume 后首 100 steps 的 loss/update parity

6.2 典型失败模式与根因

症状	常见根因	检查顺序
warmup 初期 loss spike	LR 太快、 β_2/ϵ 与梯度尺度不匹配、FP16 overflow	overflow → grad norm → update ratio → LR
训练中后期突然震荡	data/length phase change、v_t 太慢、局部 layer update 放大	per-layer v/update → phase boundary → batch stats
模型权重 norm 持续漂移	WD 太弱/太强、scale-invariant equilibrium 改变	weight norm → WD ratio → angular update
resume 后 loss 不连续	optimizer state / scheduler / scaler 未完整恢复	step counter → m/v → LR → RNG/data position
换 fused optimizer 后差异	epsilon/order/dtype/overflow 语义不完全等价	kernel math → state dtype → numerical tolerance
Muon/SOAP step 更少但 wall-time 不快	每-step preconditioning/kernel overhead	step time → TFLOPs → communication → convergence

7. 如何做一个可信的 Optimizer Ablation ?

Optimizer 是最容易出现“比较不公平”的模块之一。下面是一套面向 LLM 预训练的最小实验协议。

控制变量	要求	为什么
Model/Data/Tokenizer	完全相同	否则 loss 曲不可因于 optimizer
Tokens / FLOPs / Wall-clock	三条横轴都报告	step 数不是 compute 成本
Batch size	固定或单独 sweep	optimizer 对 gradient noise 的敏感性不同
LR + WD tuning budget	每个 optimizer 都获得公平预算	用 AdamW 的超参直接套新 optimizer 会偏置结论
Warmup/Schedule	固定 baseline + optimizer-specific best	分清 optimizer gain 与 scheduler gain
Precision/Kernel	记录 state dtype、fused kernel、offload	系可能比算法本身影更大
Scale/Horizon	至少多个模型规模 + 多训练 horizon	小模型/短训练优势可能不外推
Downstream Eval	不能只看 training loss	更快压 loss 不一定带来同等能力提升

建议实验矩阵

先用 50M-300M proxy 做 $LR \times WD \times \beta$ sweep，再在 1B 左右做 long-horizon；只有在 token efficiency、wall-clock、peak memory 与 downstream 多维度都保持优势时，再进入 multi-billion production trial。

7.1 Optimizer 与 Learning Rate Schedule 必须分开归因

η_t 决定全局时间尺度，optimizer 决定“给定 η_t 时怎样变换梯度”。很多新优化器同时更换 β schedule、weight-decay schedule 或训练 horizon，因此必须至少报告：同 schedule 对比、各自 best schedule 对比，以及 schedule-free / horizon-free 是否真的带来运营价值。下一册将单独讨论 LR Schedule。

8. Know-Why / Know-How 总结

问题	Know-Why	Know-How
为什么 AdamW 是 LLM 默认基线？	Momentum + diagonal adaptivity 在噪声/尺度异质的 Transformer 中稳健，且 WD 解耦语义清楚	先把 AdamW recipe、scheduler、clip、precision 与 state checkpoint 做到稳定，再比较新 optimizer
为什么 β_2 很关键？	它定义二阶尺度估计的记忆长度	记录 $\sqrt{v}$ 的层级统计；phase change 前后检查响应速度
为什么 optimizer 很耗显存？	AdamW 需要与参数同形的 m/v ，混合精度还常有 master/grad buffers	Adafactor、8-bit state、BF16 moments、ZeRO/Distributed Optimizer 都是在压这部分
为什么 WD 不是普通 L2？	adaptive preconditioner 会重标定进入 gradient 的 L2 项	AdamW 做 decoupled decay；按参数组设计 WD
为什么要看 update/weight ratio？	正改模型的是 $\Delta\theta$ 而不是原始 g	per-layer 监控 grad、moment、update、weight 四层尺度
为什么新 optimizer 要谨慎？	step efficiency、wall-clock、memory、kernel maturity 和成本可能互相抵消	使用统一 benchmark，多 scale、多 horizon，明成熟数据 vs 新预印本

一句话沉淀

Optimizer 的真正工程对象不是“选 Adam 还是 Muon”，而是构建一个可解释、可扩展、可恢复的 Update Control Plane：它知道每类参数如何利用历史梯度、如何限制更新尺度、如何存储状态、如何跨 GPU 分片，以及如何证明这些选择在更大规模和更长 horizon 下仍然成立。

参考资料与证据等级

ID	资料	URL	证据等级
R1	Kingma & Ba, Adam: A Method for Stochastic Optimization, 2014/ICLR 2015	https://arxiv.org/abs/1412.6980	经典/成熟
R2	Loshchilov & Hutter, Decoupled Weight Decay Regularization, ICLR 2019	https://arxiv.org/abs/1711.05101	经典/成熟
R3	Shazeer & Stern, Adafactor: Adaptive Learning Rates with Sublinear Memory Cost, 2018	https://arxiv.org/abs/1804.04235	经典/成熟
R4	You et al., Large Batch Optimization for Deep Learning: Training BERT in 76 minutes, 2019	https://arxiv.org/abs/1904.00962	经典/成熟
R5	Rajbhandari et al., ZeRO: Memory Optimizations Toward Training Trillion Parameter Models, 2019/SC 2020	https://arxiv.org/abs/1910.02054	经典/成熟
R6	Dettmers et al., 8-bit Optimizers via Block-wise Quantization, 2021	https://arxiv.org/abs/2110.02861	成熟研究
R7	Liu et al., Sophia: A Scalable Stochastic Second-order Optimizer for LM Pre-training, 2023	https://arxiv.org/abs/2305.14342	研究证据
R8	Kosson et al., Rotational Equilibrium: How Weight Decay Balances Learning Across Neural Networks, 2023/24	https://openreview.net/forum?id=FyCkPgTIXO	机制研究
R9	Vyas et al., SOAP: Improving and Stabilizing Shampoo using Adam for Language Modeling, ICLR 2025	https://proceedings.iclr.cc/paper_files/paper/2025/hash/e988664070e9591f93fdcf605f7dc623-Abstract-Conference.html	同行评审
R10	Liu et al., Muon is Scalable for LLM Training, 2025	https://arxiv.org/abs/2502.16982	大规模预印本/公开 checkpoint
R11	Pagliardini et al., The AdEMAMix Optimizer: Better, Faster, Older, 2024	https://arxiv.org/abs/2409.03137	研究证据
R12	PyTorch AdamW documentation (current)	https://docs.pytorch.org/docs/stable/generated/torch.optim.AdamW.html	官方实现

ID	资料	URL	证据等级
R13	NVIDIA Megatron Core Distributed Optimizer documentation (current)	https://docs.nvidia.com/megatron-core/developer-guide/latest/user-guide/features/dist_optimizer.html	官方实现
R14	DeepSeek-V3 Technical Report, 2024/2025	https://arxiv.org/abs/2412.19437	大规模公开训练报告
R15	The Llama 3 Herd of Models, 2024	https://arxiv.org/abs/2407.21783	大规模公开训练报告
R16	Semenov et al., Benchmarking Optimizers for Large Language Model Pretraining, 2025/26 revision	https://openreview.net/forum?id=fL9qDVnMJF	benchmark / under review
R17	Ferbach et al., Logarithmic-time Schedules for Scaling Language Models with Momentum (ADANA), 2026	https://arxiv.org/abs/2602.05298	2026 前沿预印本

证据使用说明：本册把 Adam/AdamW/Adafactor/ZeRO/PyTorch/Megatron 等成熟机制与 2025-2026 新 optimizer 工作分层处理。新方法的 speedup 数字仅代表对应论文实验条件，不应直接外推到任意模型、batch、硬件或训练 horizon。